Tasmota 插座 MQTT 网页控制台

web\mqtt-web.py · 多设备网页控制台 · 使用说明

v3.7 · 2026-09-27 · 设备 tasmota_6FCE2C（固件 15.5.0 / ESP32-C2）
新增「实时功率 / 电压」卡片：实时功率、电压、电流、功率因数、今日与累计用电

项目	内容
控制对象	Tasmota 插座：继电器开关 + 设备侧定时器
Broker	`s1.20202029.com` 1883 明文 / 8883 TLS 1.2（两端都需用户名+密码）
默认设备	`6FCE2C` → 主题 `tasmota_6FCE2C`
本册讲谁	`web\mqtt-web.py` 多设备网页控制台
工作目录	`D:\aiwork\mqtt-py\web` —— 面板、数据和本册都在这里
Python	3.13（虚拟环境 `.venv`）
依赖	`paho-mqtt`；生成本 PDF 需要 `reportlab`；跑测试可选 `pytest`

目录

1 一分钟上手
2 环境与文件清单
3 页面上能做什么
4 启动与首屏
5 多用户：注册、登录、改密码
6 设备清单与密码本
7 定时器要点
8 循环（鱼缸模式）
9 实时功率与电压
10 常用参数
11 改 MQTT 服务器（设置）
12 从手机 / 局域网里访问
13 安全提醒（重要）
14 常见问题排查
附录 A MQTT 主题映射
附录 B 本机实测注意事项

1 一分钟上手

这是一个**网页控制台**：一个文件管**一整张插座清单**，浏览器里点着用，手机也能开。

三步就能用起来：① 进入 `web\` 目录启动面板；② 浏览器打开它打印出来的地址；③ 注册第一个账户，然后把插座序列号加进清单。面板启动后**窗口要一直开着**。

```
cd D:\aiwork\mqtt-py\web
mqtt-web.py REM double-clicking the file works too
REM it prints the address to open, e.g. http://127.0.0.1:8765/
```

我想……	怎么做
开 / 关 / 翻转	左栏选中插座，点 ON / OFF / 翻转
看它在线没有	左栏 ONLINE 指示灯，清单自动轮询刷新
开 5 秒后自动关	点「脉冲」，填秒数（面板后台倒计时）
开 30 分、关 30 分，一直循环	点「循环（鱼缸模式）」那一栏，填秒数后开始（见第 8 章）
设定时器	「定时器」页：16 个槽位 + 全局 Timers 总开关（见第 7 章）
加一台插座	左栏「+ 添加序列号」，填序列号 + 备注
换 MQTT 服务器	左栏底部「设置」，保存即生效，不用重启（见第 11 章）
用手机开	看启动信息里 network 那一行（见第 12 章）

三条最容易忽略的前提，先看这里能省掉大半排查时间：

> ① **面板窗口必须一直开着**：设备轮询和脉冲倒计时都在这个进程里，关掉窗口脉冲就不会切回去了。
> ② **定时器用「设备本地时间」**，而本机时区是固件默认的 `+01:00`，不是北京时间 —— 设 08:00 会在北京 15:00 触发。先在时钟页写入 `+8:00`。
> ③ **定时器要过三道开关**：全局 `Timers` 开关 + 槽位 `Enable` + 时间到。本机全局开关初始是 `OFF`，只写槽位不会触发。

面板是**独立单文件**（约 220 KB），不 import 项目里任何东西；账户和每人一份的设备清单都写在它**自己旁边**（同一个 `web\` 目录），所以整个目录拷到别的机器上照样能跑。

2 环境与文件清单

2.1 依赖

```
cd D:\aiwork\mqtt-py

# .venv already ships paho-mqtt + pytest; rebuild them with:
.venv\Scripts\python.exe -m pip install paho-mqtt

# or with uv:
uv venv && uv add paho-mqtt
```

2.2 文件清单

要跑的东西只有 `web\` 一个目录：面板本体在里面，它运行时写的账户表、设备清单、设置文件也都在里面，本册同样生成在这里。所以整个目录拷到别的机器上照样能跑，不用到处找配置文件。

```
D:\aiwork\mqtt-py\ <- the project root
|-- web\ <- mqtt-web.py and the files it writes
| '-- the web manual is written here too
|-- build-web\ <- the generator, only to rebuild
|-- .venv\ <- Python 3.13 environment
'-- .workbuddy\ <- project memory folder, keep it
```

要用的文件（都在 `web\` 里）

文件	作用
`mqtt-web.py`	**网页控制台本体**：单文件管理**一整张设备清单** —— 浏览器访问 `http://127.0.0.1:8765/`，局域网内把 127.0.0.1 换成本机 IP 即可（见第 12 章）
`mqtt-web-users.json`	**账户表**：用户名 / 邮箱 / 加盐哈希（**没有明文密码**），注册第一位用户后出现（见第 5 章）
`mqtt-web-users\*.json`	**每人一份自己的设备清单**，互不干扰（见第 5 章）
`mqtt-web-cycle.json`	**循环的配置记忆**：每台插座记住的开 / 关时长、先开还是先关、断电后要不要自己接着跑（见 §8.4）。多人模式每人一份，名为 `<account>-cycle.json`
`mqtt-web-vault.json`	**设备序列号原始密码本**：管理员维护的「序列号 → 原始密码」对照表，普通用户添加设备时要按它验证；**里面是明文**，要当心和插座一样保管（见 §6.2）
`mqtt-web-settings.json`	**MQTT 服务器设置**（地址 / 端口 / 用户名 / 密码等），只有在页面里点过「保存并应用」后才会出现（见第 11 章）
`mqtt-web-devices.json.adopted`	升级留下来的**旧共用清单**（第一个账户接管它之后改名保留，可删）

这些 json 都在 `web\` 里、**挨着面板自己**，所以整个目录拷到别的机器上照样能跑；旧版放在根目录的 `mqtt-web-devices.json` 会在第一次注册时被接管过去。

工作目录里还有

文件	作用
`build-web/`	**打造上面这些文件的工具，日常用不到**（见 2.3）

文件	作用
`.venv/`	虚拟环境（Python 3.13.12）
`.workbuddy/`	项目记忆目录，勿删

2.3 build-web/ 里有什么

文件	作用
`build-web/engine.py`	引擎源码 —— 网页面板的正文就是从它复制来的
`build-web/webhub.py`	多设备网页控制台（`mqtt-web.py`）的源码（make_shortcuts.py --multi 生成）
`build-web/webapp.py`	单设备面板那部分源码（按需生成时用：make_shortcuts.py 设备名 --web）
`build-web/make_shortcuts.py`	生成器：把引擎和面板拼成一个文件（`--> web\mqtt-web.py`）。改了源码必须重跑它
`build-web/selftest.py`	离线回归测试，不需要 broker（顺带校验面板是否过期）
`build-web/make_manual.py`	生成本 PDF 的脚本

> 跑自检：`python build-web\selftest.py` 或 `pytest build-web\selftest.py` —— 都不联网，秒级完成。

3 页面上能做什么

`web\` 目录里的 **网页控制台** `mqtt-web.py`
不是「一台设备开一个窗口」，它一个页面管理**一整张设备清单**：左栏列出所有插座（序列号 + 备注 + 在线状态），点谁控制谁；序列号可以随时**添加、改名、改备注、删除。**浏览器访问 `http://127.0.0.1:8765/` 即可。

它是**独立单文件**（约 220 KB），不 import
项目里任何东西：引擎代码已经内联在里面，所以设备怪癖的处理（POWER0
回退、超时、权威回读）全都带着走。账户和每人一份的设备清单都放在它**自己旁边**（同一个 `web\` 目录）。

面板**默认监听所有网卡**，所以同一个 Wi-Fi /
局域网里的手机、平板、另一台电脑都能直接开；只有本机想独享时再加 `--localhost`（见第 12 章）。

页面上能做什么	说明
设备清单（左栏）	全部序列号 + 备注 + 在线状态；添加 / 删除 / 改名 / 改备注都在这里
继电器 / 脉冲	ON / OFF / 翻转 + ONLINE 指示灯，两者**同一张卡片**：上面一行是开关，下面一行是脉冲（保持 N 秒后自动切到另一态，倒计时在面板后台）
实时功率 / 电压	读插座自己的计量芯片：实时功率、电压、电流、功率因数、视在 / 无功、今日与累计用电（见第 9 章）
循环（鱼缸模式）	开 N 秒 / 关 M 秒一直重复。写进插座自己的规则里，关掉电脑也照跑（见第 8 章）
循环的配置记忆	记住上次设的开 / 关时长；断电后来电点「按记忆重启」就接着跑（见 §8.4）
定时器	16 个槽位列表 + 改写任一槽位 + 全局 `Timers` 总开关

页面上能做什么	说明
时钟与时区	显示设备 UTC / 本地 / 时区，一键写入 `+8:00`
改 MQTT 服务器	左栏底部「设置」：地址 / 端口 / TLS / 用户名 / 密码，保存即生效
密码本（只有管理员看得见）	左栏底部「密码本」：维护「序列号 → 原始密码」。本子里有内容之后，添加设备必须填对密码才行（见 §6.2）
手机 / 小屏	窄屏自动换一套布局：设备清单变成一条能左右滑的横条、卡片排成一列、按钮和输入框放大到手指大小（见第 12 章）

4 启动与首屏

在 `D:\aiwork\mqtt-py\web` 目录里启动（双击 `mqtt-web.py` 也行，它是自包含的）：

```
D:\aiwork\mqtt-py\web>mqtt-web.py
mqtt-py web panel -- 0 signed in out of 0 account(s)
broker : s1.20202029.com:1883
accounts: D:\aiwork\mqtt-py\web\mqtt-web-users.json (0)
lists : D:\aiwork\mqtt-py\web\mqtt-web-users\<account>.json (one per user)
nobody yet -- open the page and register the first account
settings: D:\aiwork\mqtt-py\web\mqtt-web-settings.json (none yet)
vault : ...\web\mqtt-web-vault.json (0 entries -- empty, so adds
are NOT checked; the first account to register is the admin)
listen : 0.0.0.0:8765
local : http://127.0.0.1:8765/
network : http://192.168.1.23:8765/ <- use THIS from a phone
NOTE : open to your whole network, so anyone reaching this port gets the
LOGIN page -- not the devices. Accounts are what they have to get past.

Ctrl-C to stop.
```

打开 `http://127.0.0.1:8765/`

先看到**登录页**：一个人都没有时会**直接停在「注册新账户」**那一面，注册完自动登录进面板（见第 5 章）。

浏览器打开 `http://127.0.0.1:8765/` 后，点 **「+ 添加序列号」**：填序列号（`6FCE2C`、`tasmota_6FCE2C`、带 `DVES_` 前缀都行，面板会自动归一化成短 ID）和一行备注（如「客厅插座」），保存后面板立刻探测一次该设备。设备暂时不在线也会**照常入库**——方便先把清单建好、硬件随后到位。

面板窗口必须一直开着：设备轮询和脉冲倒计时都在这个进程里，关掉窗口，脉冲就不会在计时结束时切回去了。

5 多用户：注册、登录、改密码

面板**默认就是多人模式**：第一次打开是**登录页**，没有账户就点「注册新账户」—— 用户名 + 邮箱 + 密码（至少 8 位），密码要在「**再输一次密码**」框里**再填一遍**才算数，注册完**直接就登进去了**。以后每次打开都要登录，只有登录后**自己的那份设备清单**才会出现。

要做什么	在哪
注册第一个账户	登录页 →「注册新账户」（没有账户时它会直接停在这一面）
登录	登录页，填用户名 + 密码
改密码 / 改邮箱	进面板后左栏顶部 **「账户」** 按钮
退出登录	左栏顶部 **「退出」** 按钮
增删设备序列号	和以前一样，只是**改的是你自己那份清单**

几个刻意这么设计的点：

- **改密码要填旧密码**：光有登录态不够。改完会**把这个账户其他地方的登录踢下线**（当前窗口保留），所以密码改了，别人拿旧会话也进不去。
- **「用户名不存在」和「密码不对」报同一句错**：分开报等于免费告诉别人这里有哪些账户。连错 8 次会锁 15 分钟，防止一直猜。
- **每人一份清单文件**：存在 `web\mqtt-web-users\` 目录里（用户名就是文件名）。Alice 加的插座 Bob 看不见，反之亦然。
- **会话只在内存里**：重启面板所有人都得重新登录，12 小时没动静也会自动掉线（页面开着就算活跃）。
- **第一个注册的人是管理员**：他会多一个左栏底部的「**密码本**」按钮，用来维护设备序列号和原始密码（见 §6.2）。别人注册得再早也没用 —— 角色记在账户表里，升级时面板会把**最早的那个**旧账户自动补成管理员。

升级不用重来：以前那份单人清单 `mqtt-web-devices.json` 会由**第一个注册的人**接管（序列号全部搬进他自己的清单），旧文件改名保留成 `.adopted` —— **不删任何东西**，万一搬错了还能翻回去。

账户表 `mqtt-web-users.json` 长这样（UTF-8）：

```
{
"version": 1,
"users": [
{"name": "alice", "email": "alice@example.com",
"salt": "9f2c... <- random 16 bytes per user",
"hash": "e7b1... <- PBKDF2-HMAC-SHA256",
"rounds": 200000, "created": "2026-09-27T10:12:00",
"role": "admin"} <- admin | user (first account)
]
}
```

密码**永远不以明文落地**：每人一把随机盐，所以两个人选同一个密码，文件里看起来也不一样。这个文件**解析不了就报错不覆盖**，和清单文件一个规矩 —— 它是所有人的密码，删坏了谁都登不进来。

参数	说明
`--no-register`	关掉注册：家里几个人都建完之后加上，外人就建不了账户了
`--no-auth`	回到**不用登录**的老样子：全站共用一份 `mqtt-web-devices.json`（单人用最省事）
`--users FILE`	换一个账户表路径（默认在面板旁边）

参数	说明
`--lists DIR`	换一个「每人一份清单」的目录

> **注意：注册是默认开放的**：因为面板默认局域网可访问（见第 12 章），同网的任何人都能建账户、建自己的清单。家里几台设备都配好之后，建议**加 `--no-register` 重启**，并同时用 `--token SECRET` 挡一道。忘了密码没法找回 —— 停掉面板删掉 `mqtt-web-users.json` 就能重来（会连账户一起清掉）。

6 设备清单与密码本

6.1 清单文件长什么样

多人模式下，每个账户的清单是 `web\mqtt-web-users\` 目录里**属于自己的那一个文件**（`alice.json`、`bob.json` …），字段和下面说的完全一样；`mqtt-web-devices.json` 用 `--no-auth` 时才会用得上。下面把它统称「清单文件」。

字段	类型	含义
`id`	字符串	序列号，1~32 位，字母 / 数字 / `_` / `-`
`note`	字符串	备注，一行纯文本，可为空（中文没问题）
`added`	字符串	入库时间（UTC），面板自动填写

清单就是这个小文件，**手工编辑完全没问题**（面板停着的时候改最稳）。面板每次保存都是原子写（先写临时文件再改名），并留一份 `.bak` 备份；如果文件被改坏到解析不了，面板会报错并**拒绝覆盖**它 —— 坏文件起跑时是空清单，原文件原样保留，修好再重启即可。

真实形状（面板读写一律 UTF-8）：

```
{
"version": 1,
"devices": [
{"id": "6FCE2C", "note": "living-room plug",
"added": "2026-09-26T11:00:00"}
]
}
```

上限默认 32 台（每台一条 MQTT 连接），用 `--max-devices` 调整。在页面上「删除」只是**忘掉这台设备的记录和备注**，不会向设备发任何东西。

6.2 设备序列号原始密码本（管理员）

「添加序列号」是这个面板里唯一一个能让面板去指挥**陌生硬件**的动作，所以它也是唯一一个需要第二个人点头的动作：管理员在左栏底部点**「密码本」**，把「序列号 → 原始密码」一条条登记进去（也可以加一行备注，例如「客厅空调」）。

本子的状态	添加设备时会怎样
空的	不校验，谁都能添加 —— 否则第一台设备永远加不进去
有内容	所有人（管理员也一样）都必须填**对得上的序列号和原始密码**，否则被拒，清单不会变

判定只认两样：序列号（**大小写不敏感**，`6fce2c` 也能找到 `6FCE2C`）和原始密码。序列号不在本子里 → 提示「请先联系管理员建档」；序列号在、密码不对 → 提示「原始密码不对」。两种情况**都不会**顺带告诉你这台设备是不是已经在谁的清单里 —— 那是别人清单上的事。

密码本弹窗里每行的密码默认是**圆点**，点「显示」才看得见；「编辑」可以改密码或备注 —— **密码留空就只改备注**，绝不会被清空；「删除」要确认一次。删掉一条之后，普通用户就不能再添加那个序列号了，已经加进清单的设备不受影响。

只有管理员能打开这个窗口，而且服务端也拦：普通用户就算自己拼出 `/api/vault`，拿到的也只是 403。页面把按钮藏起来只是顺手，不是防线。

> **注意：这个文件里存的是明文原始密码。**它和插座本身一样敏感 —— 属于要一起备份、也一起保密的东西。面板不会把它发给非管理员，也不会塞进 `/api/devices` 这类人人都会读到的返回里。

管理员是谁：**第一个注册的账户**就是管理员（登录页也是这么写的）。旧版本建的账户表里没有角色，面板加载时会自动把**最早的那个**账户补成管理员并把结果写回文件。如果补错了人，当前管理员可以调一次 `POST /api/auth/role` 把角色交出去（最后一个管理员不能把自己降级）。

7 定时器要点

定时器是**存在插座里、由设备自己执行**的，所以面板设完就可以关掉浏览器 ——
但它有三个前提，缺一个就不会响。

7.1 三道开关，缺一不可

层	是什么	在面板上怎么开
全局	设备的 `Timers` 总开关	定时器页顶部的总开关，置 ON
槽位	该槽位自己的 `Enable` 字段	编辑槽位时勾上「启用」
时间	设备本地时间到点	见 7.2，先把时区改成 `+8:00`

> 本机全局开关出货状态是 `OFF`。只写槽位、不开全局开关，看起来「设置成功了」但永远不会触发 ——
> 这是最常见的坑。

7.2 时区：先改，再设定时器

定时器按**设备本地时间**触发，而本机固件默认时区是 `+01:00`、不是北京时间 —— 设 08:00 会在北京时间 15:00
才响。时钟页会列出设备 UTC、本地时间和当前时区，点一下写入 `+8:00`
即可（中国没有夏令时，写固定偏移就行）。

> **注意：本机固件只认「`+8:00`」这种写法**：带前导零的 `+08:00` 会被**静默拒绝** —— 设备回一个格式完全正常的 `{"Timezone":"+00:00"}`，看不出报错，时区却回落成
> UTC。面板写入后会**回读校验**，失败会明确报错而不是假装成功。

7.3 星期掩码 Days（最容易错的地方）

掩码共 7 位，**第 1 位是周日**，顺序固定 `S M T W T F S`；`0` 和 `-`
都表示「关」。面板上勾选星期时会自动换算，但手工填掩码时要记住周日在最前。

位序	1	2	3	4	5	6	7
代表	日	一	二	三	四	五	六
周一~周五	0	1	1	1	1	1	0
周末	1	0	0	0	0	0	1
每天	1	1	1	1	1	1	1
错误写法	1	1	1	1	1	0	0

> 最后一行 `1111100` 是**周日~周四**，不是周一~周五。

7.4 槽位字段

字段	含义与取值
`Enable`	槽位开关：1 启用 / 0 停用
`Mode`	时间基准：固定时刻 / 日出 / 日落
`Time`	触发时刻（00:00~23:59）；日出/日落模式是相对偏移，小时最大 11
`Window`	随机抖动分钟数 0~15（避免所有设备同时动作）

字段	含义与取值
`Days`	7 位星期掩码，顺序 `SMTWTFS`，**周日在第 1 位**
`Repeat`	每天重复，或只触发一次并在触发后自动停用
`Output`	驱动哪一路输出 / 继电器，1~16，本机填 1
`Action`	关 / 开 / 切换 / 触发规则

共 16 个槽位。改写槽位时面板**先读该槽位现值、只合并你改的字段、再回写完整 JSON**，所以改一个字段不会动到同槽位的其它字段。

8 循环（鱼缸模式）

想要的是「开一段时间、关一段时间，一直这么下去」—— 鱼缸灯、气泵、定时通风都是这个节奏。**定时器做不到**：一个槽位一天只响一次，它不会自己交替。能做到的是设备里的**规则（Rules）**，面板上的「循环」卡片就是把这套规则写好、写好就交给插座去跑。

8.1 怎么用

步骤	做什么
① 填秒数	「秒开」和「秒关」两个框。面板给了 30 秒 / 5 分 / 15 分 / 30 分 / 1 时 / 10 分+50 分几个快捷键，点一下两个框一起填好
② 选先做什么	「先开 ON」= 立刻通电，开满后关；「先关 OFF」= 立刻断电，关满后开
③ 开始循环	点「开始循环」。几秒后状态行会显示当前是开着还是关着、还有多少秒切换
④ 想停就停	点「停止」。规则留在插座里但被停用，**继电器停在当前状态、不会被强行改**

> 循环是**写进插座里的**，不是写在电脑上：关掉浏览器、关掉这台电脑、断掉 Wi-Fi，它照跑。这也是它和「脉冲」最大的区别 —— 脉冲的倒计时在面板进程里，窗口一关就不切回去了。

8.2 它到底往设备里写了什么

一句话：两条规则互相给对方上闹钟。写在设备的 `Rule1` 里，长这样（以开 10 分 / 关 10 分为例）：

```
ON Rules#Timer=1 DO Backlog Power1 ON; RuleTimer2 600 ENDON
ON Rules#Timer=2 DO Backlog Power1 OFF; RuleTimer1 600 ENDON
```

第一条：定时器 1 到点 → 通电，同时给定时器 2 上 600 秒（这段是「开」多久）。第二条：定时器 2 到点 → 断电，同时给定时器 1 上 600 秒（这段是「关」多久）。两个闹钟轮流生效，就构成了无限循环。面板启动时再补一条 `Backlog Power1 ON; RuleTimer2 600`，把第一段立刻兑现。

> **注意：顺序别记反 —— 定时器 2 是「开多久」，定时器 1 是「关多久」**：每个闹钟管的是**另一段**的长度。面板已经处理好了，这个提醒是给手工在控制台里改的人准备的。

8.3 会看到什么

显示	意思
没在跑	规则没启用
没在跑（规则还在插座里，已停用）	点过停止；再点开始就能接着跑
没在跑（规则还在插座里 —— 断电后计时不恢复）	像是断过电：设置还在，只是没人按启动键（见 §8.4）
Rule1 被别的规则占用	插座的 `Rule1` 里有别人写的规则，面板不会去动它
循环中：开 … / 关 … 现在是开着	正在「开」这一段
N 秒后关 / N 秒后开	设备自己报的剩余秒数，每秒在走

剩余秒数是**从设备读回来的**（`RuleTimer1` 会回报 T1~T8 各自的剩余值），不是面板自己数的，所以刷新页面、换一台电脑看，数字都对得上。

8.4 配置记忆：记住上次设的开 / 关时长

循环有两类东西可以「记住」，**面板只记第一类**：

类别	记不记	为什么
配置（开多久 / 关多久 / 先开还是先关）	**会记**	这是你自己设的、不会自己变的东西。记下来，明天开面板、换台电脑打开，数字都还在，不用重新填
进度（这一段还剩几秒）	**不记**	进度在插座的定时器里，**断电就归零**。记下来只会变成一个和设备对不上的假数字，所以面板不猜，直接告诉你「规则还在，但没在跑」

插座自己断电后留住了什么（这是 Tasmota 的行为，不是面板的功能）：

插座里的东西	断电后
Wi-Fi、设备名称、模块配置	**保留**
普通定时器 `Timer`（16 个槽位）	**保留**
规则 `Rules`（循环就写在这里）	**保留**
上一次继电器是开还是关	**默认保留**
定时器**当前走到第几秒**（`RuleTimer`）	**不保留** —— 归零，循环停在那儿不动

所以停电再来电时，你会看到状态行写着**「没在跑（规则还在插座里 —— 断电后计时不恢复）」**：设置一点没丢，只是没人替它按下启动键。

8.5 记忆怎么用

按钮 / 选项	做什么
记住这组数字	把上面两个框里的开 / 关秒数和「先开 / 先关」存下来，**不会**开始循环 —— 先在桌上配好、到鱼缸那边再启动
用记忆填充	把记住的数字填回上面两个框（也不开始）
按记忆重启	按记住的设置**立刻开始**循环。停电后来电、或者刚换了一台电脑开面板，点这一个就行
清除记忆	忘掉这台插座记住的设置
断电后自动接着跑	勾上后，只要面板开着、并且上次是「在跑」的状态被停掉的（不是你手动点停止的），它检测到「规则还在、计时归零」就自动重启一次

> 自动重启是**每台设备只试一次**：失败了会把原因写在卡片下面的提示行里，不会反复重试。手动点过「停止」的循环**不会**被自动拉起来 —— 「停止」是你下的决定，面板不替你反悔。

记忆写在 `mqtt-web-cycle.json` 里（多人模式下每人一份，叫 `<account>-cycle.json`）：纯文本 JSON、原子写入、改前自动留一份 `.bak`，看不懂的文件只会报错、不会覆盖。

8.6 几条要注意的

- 取值范围 1 ~ 86400 秒（24 小时）。更长的会被拒绝。
- **插座断电重启后循环不会自己恢复**：规则还在，但定时器不持久，重新插电后要再点一次「开始循环」—— 或者点「按记忆重启」用上次那组数字，也可以勾上「断电后自动接着跑」让面板帮你按（见 §8.4、§8.5）。
- 只有 `Rule1` 被这条循环占用；如果你自己在控制台里给 `Rule1` 写过别的规则，面板第一次点开始时会**先提示**，确认后才覆盖 —— 不确认就什么都不动。
- 停止时**不会**顺手把继电器扳到某一态：鱼缸的泵停在哪一态，应该由你决定，不该由「停止」这个动作替你决定。
- 循环和定时器互不影响：定时器那 16 个槽位照常可用，两者各走各的。

9 实时功率与电压

只要插座里有**计量芯片**，面板就会在「实时功率 / 电压」卡片里一直显示它的读数 ——
这是插座自己测出来的数字，面板只是读回来，中间不做任何换算。

9.1 卡片上都有什么

位置	字段	单位	说明
大字	功率	W	当前正在消耗的功率
九宫格	电压	V	市电电压，判断是否欠压 / 过压
	电流	A	当前电流
	功率因数	—	0 ~ 1；纯阻性负载接近 1
	视在 / 无功	VA	视在功率 / 无功功率，仅部分芯片上报
	今日用电	kWh	从设备当天 0 点算起
	累计用电	kWh	设备自己的总电量表
进度条	—	—	相对**这台设备本次打开以来见过的最大功率**；靠近满格变红，只是示意「比刚才忙」，不是额定功率

字段是**按设备挑**的：不同芯片（`ENERGY`、`ENERGY1`、`BL0942`、`PZEM0xx`…）上报的键名不一样，面板认得 `Today` 与 `EnergyToday` 两种写法，认不出的就当没有这个字段，不会瞎猜一个
0。所以有的插座只有功率和电压，有的六个小格都满。

9.2 三种「没有读数」，含义完全不同

卡片上显示	什么时候出现	该怎么办
数字 + 「N 秒前」	一切正常，N 是这次读数到现在的秒数	不用管；秒数在两次读取之间会自己往上走
大字 **n/a** +「此设备无功率计量」	插座回话了，但**它没有计量芯片**（或固件没打开传感器）	这是正常结论，不是故障。面板此后很少再去问它
「未读到读数（上次 N 秒前）」	插座没回话（离线、掉线、broker 不通）	数字留在屏幕上并标出年龄；恢复通信后会自动变新

> **为什么无计量不写 0 W：**「没有计量芯片」和「负载正好是 0 瓦」在屏幕上长得一模一样，但一个是「测不了」、一个是「真的没用电」。面板宁可用文字说清楚，也不给你一个设备从来没说过的数字。

9.3 它多久读一次：`--energy`

功率是**独立的一条节拍**，和继电器状态那条分开：按一下开关不会改变用电量，所以它不跟着按钮的快读窗口走，而是自己每 `--energy` 秒读一次（默认 **5** 秒）。

为什么不是更快：**同一台插座的每一次读取都排在同一条队列里**（继电器状态、定时器、循环、功率共用它）。把 `--energy` 调到比 `--refresh` 还小，并不会让功率更实时，只会让继电器状态读得更慢 ——
读功率的请求把队占了。默认 5 秒对继电器 8 秒，已经足够「实时」了。

- 读不到时它**自己退让**：失败后先等 20 秒，再失败翻倍，最多退到 120 秒；读到一次就立刻回到 5 秒。
- **没有计量芯片的插座**只问一次：确认「无计量」之后就把节拍停在最长间隔，整张清单都是这种插座时几乎不产生流量。

- 切设备时卡片清空并**立刻重读**，不会把上一台的瓦数留在新序列号下面；`--read-only`（只读）模式下照常工作。

面板读的是设备自己的传感器块，用的是 `Status 8` 这条命令（回在 `STATUS8` 上）。注意：`StatusSNS` 只是那段载荷里的**键名**，**不是**能发出去的指令 —— 发它设备会回「Unknown」。所以本手册只说 `Status 8`。

调大调小都在启动时定：`mqtt-web.py --energy 5`（1 ~ 300 秒），想恢复默认就写 5。

10 常用参数

参数	默认	说明
`--port 8080`	`8765`	面板自己的监听端口；被别的程序占着时换这个
`--listen 0.0.0.0`	`0.0.0.0`	绑到哪张网卡；`0.0.0.0` = 所有网卡，局域网都能访问（**默认值**，见第 11、12 章）
`--localhost`	关	只绑 `127.0.0.1`，**只有本机能开**（笔记本 / 公用网络用这个）
`--list FILE`	`mqtt-web-devices.json`	换一个清单文件（只在 `--no-auth` 下有用；多人模式每人一份，见第 5 章）
`--no-auth`	关	免登录：回到「全站共用一份清单」的老样子（见第 5 章）
`--no-register`	关	关掉注册：已有账户照常登录，但建不了新的（见第 5 章）
`--users FILE`	`mqtt-web-users.json`	换一个账户表路径（见第 5 章）
`--lists DIR`	`mqtt-web-users\`	换一个「每人一份清单」的目录（见第 5 章）
`--max-devices 64`	`32`	最多同时管理多少台设备
`--settings FILE`	`mqtt-web-settings.json`	换一个服务器设置文件（默认和清单放在一起）
`--cycle FILE`	`mqtt-web-cycle.json`	换一个「循环配置记忆」文件（只在 `--no-auth` 下有用；多人模式每人一份，见 §8.4）
`--vault FILE`	`mqtt-web-vault.json`	换一个设备密码本文件；本子里有内容时，添加设备要按它验证（见 §6.2）
`--refresh 15`	`8`	页面多久读一次**你正在看的那台**插座（秒）；这是唯一按节拍主动碰设备的设置，服务器慢 / 多个标签页看同一台时调大它
`--sweep 90`	`45`	后台清单多久刷新**全部设备**一次（秒），用来更新左栏 ONLINE 灯；读的是面板缓存，不额外打扰设备
`--energy 5`	`5`	页面多久读一次**计量芯片**：功率 / 电压 / 电流 / 电量（秒）；这是一条**独立于** `--refresh` **的节拍**，1 ~ 300 秒。调得比 `--refresh` 还小只会让继电器状态读得更慢（见 §9.3）
`--token SECRET`	无	要求以 `?token=SECRET` 打开页面
`--read-only`	关	只读模式：页面能看到所有状态，但任何写入都被拒绝
`--open`	关	启动后自动用默认浏览器打开页面
`--log-requests`	关	把每条 HTTP 请求打进面板日志

引擎的全部 MQTT 参数它也照收，只是旗标改了名：`--host` 变 `--mqtt-host`、`--port` 变 `--mqtt-port`——面板自己的 `--port` 留给了监听端口，两边互不干扰。

上面那些 `.json` 文件默认都写在**面板自己所在的** `web\` **目录**里，路径都是相对面板算的 —— 所以把整个 `web\` 拷到另一台机器，账户、清单、服务器设置会一起跟着走。

页面多久读一次状态：默认**每 8 秒**读一次你正在看的那台插座（`--refresh` 可改）—— 继电器状态不是走势图，8 秒够用；而且这个节拍刻意取得比一次读取略慢，免得请求自己叠在自己身上。三个细节：**按任意按钮后那几秒会变快**（1 秒一次、持续 6 秒，让你马上看到结果）；**读失败会退避**（先 15 秒、逐次加倍、最多 60 秒），成功一次立刻回到 8 秒；**同一次读取不会并发发两条**（上一条没回来就不再发）。左栏清单另由后台线程每 45 秒刷一遍（`--sweep` 可改），那是面板缓存，不额外碰设备。

端口是独占的：同一个端口上只允许有一个面板在跑。不小心启动第二个时，它会立刻报错退出，并提示去看是谁占着：

```
ERROR: cannot listen on 0.0.0.0:8765 -> [WinError 10048] ...
another copy of this panel already owns that port;
open it, or use --port 8766
find the owner: netstat -ano | findstr :8765
```

按提示用 `netstat -ano` 查出占用者的 PID，用 `Stop-Process -Id <PID> -Force` 结束它，或者干脆换 `--port 8766`。这条保护在 Windows 上尤其重要 —— Windows 的 `SO_REUSEADDR` 语义允许两个进程**同时监听同一个端口**而不报错，两个面板都显示「正常」，但新连接被内核随机分给其中一个，页面状态时对时错。面板现在用 `SO_EXCLUSIVEADDRUSE` 把这个洞堵上了。

11 改 MQTT 服务器（设置）

左栏底部的 **「设置」** 按钮打开服务器设置：地址、端口、TLS、用户名、密码、每次请求的超时。点 **「保存并应用」** 后面板立刻把值写进 `mqtt-web-settings.json`（和清单文件放在同一个目录），并**断开所有已建立的 MQTT 连接** —— 下一次读取就用新地址重连，**不需要重启面板**。

按钮	做什么
测试连接	拿**当前填在表单里**的值开一条临时连接，成不成都直说；**不用先保存**，正好用来在改之前先试
保存并应用	写进文件、立刻生效、断开旧连接
恢复启动值	删掉设置文件，回到命令行（或默认值）给的那一套

填错了不用猜：「测试连接」会把**端口不对**（连不上）和**用户名/密码不对**（服务器拒绝连接）分开报，两种修法不一样。

设置文件长这样（UTF-8，可以直接手改）：

```
{
"version": 1,
"host": "s1.20202029.com",
"port": 1883,
"tls": false,
"user": "mqtt",
"password": "********",
"timeout": 6.0
}
```

两个坑，先说清楚：

- **密码不回传**：服务端只告诉页面「有没有存密码」，所以那一栏打开时**永远是空的**，**留空 = 不改**。要真的清掉，得勾上「保存时清除已保存的密码」—— 否则光是打开设置再点一次保存，就会把密码清成空白。
- **密码在文件里是明文**：把这个文件当密码看待，别随手发给别人。

保存过的设置**优先级高于命令行**：下次启动时若文件里的值和参数不一致，面板会在启动信息里写出被覆盖的是哪几项；想改回命令行那条路，用「恢复启动值」或直接删掉 `mqtt-web-settings.json` 即可。

面板**自己**的地址和端口不在这个对话框里 —— 那属于监听套接字，服务期间挪不动，请用 `--listen` / `--port` 重启。

12 从手机 / 局域网里访问

面板默认绑
`0.0.0.0`（本机**所有**网卡），所以不用加任何参数，局域网里的设备就能开。启动时它会把**能用的地址全列出来** —— 因为 `0.0.0.0` 是通配符，没有任何人能在浏览器里输入这四个零：

```
listen : 0.0.0.0:8765
local : http://127.0.0.1:8765/ # this machine
network : http://192.168.1.23:8765/ # phone, another PC
network : http://192.168.56.1:8765/ # one line per network card
```

把 **network 那一行**敲进手机浏览器即可，前提是手机和这台电脑在**同一个局域网**（同一个路由器，别连着访客网络）。面板里的「设置」对话框底部也会同一份地址列出来，不用回头看命令行窗口。

手机上的布局是自动换的：窗口一窄（760 像素以下，手机竖屏都属于这一档）左栏就挪到顶部、设备清单变成一条**能左右滑**的横条，卡片排成一列，按钮和输入框放大到手指点得准的大小。不用装 App、也不用改任何设置 —— 同一份页面，宽窗口就是电脑布局，窄窗口就是手机布局；手机横屏或只占半屏也一样会自动切换。

现象	原因 / 怎么办
手机打不开，一直转圈	多半是**Windows 防火墙**拦了入站连接：放行 `8765` 端口，或者临时关掉专用网络防火墙
换路由器后地址变了	正常：地址是路由器 DHCP 发的。看启动信息里新的 `network` 行即可
只想本机用	加 `--localhost`，只绑 `127.0.0.1`，外部连不上（ refusing connect ）
要从公司 / 外网访问	**不要直接转发这个端口**：面板没有 HTTPS、token 明文，请放到反向代理后面（见第 13 章）

放行端口（管理员 PowerShell，只在防火墙开着时需要）：

```
netsh advfirewall firewall add rule name="mqtt-web 8765" dir=in action=allow protocol=TCP
localport=8765
```

查本机 IP 也可以自己来：`ipconfig`，找那张在用的网卡的 IPv4 地址。

13 安全提醒（重要）

> **注意：面板默认绑 0.0.0.0**，也就是**同一个网络里任何人都能开这个页面**：能开关清单里的**每一个**插座，也能看到整份清单。家用路由器下若做了端口转发，还可能被外网扫到。在公用网 / 公司网请加 `--localhost`；确实要长期开在局域网，**同时**加 `--token SECRET`；只想自己用就加 `--localhost` 退回本机。面板**没有 HTTPS**，token 是明文传输的，只适合可信的内网；要从外网用，请放在带 HTTPS 的反向代理后面，而不是直接把端口暴露出去。

另两个已知行为：页面上的「设备时钟」显示的是设备自己的读数（`Status 7` 返回的 `UTC` 与 `Local`），不是本机时间；改写定时器槽位时，面板先读该槽位现值、只合并你改的字段、再回写完整 JSON ——改一个字段不会动到同槽位的其他字段。

14 常见问题排查

现象	原因	处理
页面打不开，一直转圈	面板没启动，或端口被别的程序占着	看启动窗口里的 `listen` 行；`netstat -ano` 查占用者，或换 `--port 8766`（见第 10 章）
登进去了，左栏一台设备都没有	还没添加序列号（新账户的清单本来就是空的）	点左栏「+ 添加序列号」，填序列号 + 备注
加了设备却一直显示 OFFLINE	序列号写错、设备断电，或 MQTT 服务器地址不对	核对序列号（`6FCE2C` / `tasmota_6FCE2C` 都行）；再到「设置」里点「测试连接」，看服务器本身通不通（见第 11 章）
点 ON / OFF 没反应	设备不在线，或页面还没轮询到新状态	先看左栏 ONLINE 灯；灯是绿的就等几秒，列表会自动刷新
定时器设好了却不触发	三道开关没全开，或时区不对	见第 7 章：全局 `Timers` 置 ON、槽位启用、时区改成 `+8:00`
改了服务器地址后连不上	端口不对（连不上）和账号不对（被拒绝）是两回事	在「设置」里点「测试连接」，它会把这两种分开报（见第 11 章）
循环跑着跑着停了	插座断过电：规则还在，但定时器不会自己恢复	点「按记忆重启」用上次那组数字接着跑（见 §8.4、§8.5）
换台电脑开面板，循环那两个框是空的	记忆文件跟着面板走：它在 `web\` 目录里，没一起拷过去	把 `mqtt-web-cycle.json` 一起拷过去，或者在新机器上重新点一次「记住这组数字」
点开始循环提示 Rule1 被占用	插座的 Rule1 里已经有别的规则（自己写的或别的工具写的）	确认要覆盖就再点一次开始；不想动它就取消
状态读得很慢，或者偶尔提示 `no reply to `Status 0``	走公网 MQTT 服务器，一次查询本来就要零点几秒；偶尔丢一个包，或者中间那条连接断过（服务器重启、Wi-Fi 掉过）	面板每次读取会自己重试一次，连接断了会自己发现并重连，一般一两轮就恢复；一直不好就去「设置」里点「测试连接」，或把 `--timeout` 调大、把 `--refresh` 也调大（读慢一点）
忘了密码，登不进去	密码没有找回功能（只存了加盐哈希）	停掉面板、删掉 `mqtt-web-users.json` 重来（会连所有账户一起清掉）
加设备提示「不在管理员维护的设备密码本里」	管理员在密码本里登记过内容，所以添加设备开始校验；这台还没建档	找管理员在「密码本」里加一行（序列号 + 原始密码），再添加（见 §6.2）
加设备提示「原始密码不对」	序列号对上了，填的密码和密码本里记的不一样	核对设备标签上的原始密码；确实记错了就让管理员在「编辑」里改掉那一条
左栏看不到「密码本」按钮	只有管理员看得见（第一个注册的账户）	需要管密码本就找管理员；确实该自己管，让当前管理员调一次 `POST /api/auth/role` 移交角色
手机打不开，电脑能开	防火墙拦了入站连接，或手机不在同一个局域网	见第 12 章：放行端口 / 用 network 那一行的地址
功率卡片显示 `n/a` + 「此设备无功率计量」	这台插座没有计量芯片，或固件没打开传感器	不是故障：面板读的是插座自己的计量芯片，没有芯片就没有数字（见 §9.2）

现象	原因	处理
功率卡片写着「未读到读数（上次 N 秒前）」	插座没回话（掉线 / 离线 / broker 不通）	数字留着并标出年龄；看左栏 ONLINE 灯，恢复通信后自动变新（见 §9.2）
有电压，但功率 / 电流 / 累计用电都是 0	插座只回了传感器块的一部分；最常见的原因是负载确实没在用电，其次才是芯片没算完第一轮	先确认插座上真的插着正在工作的负载（电压在、电流是 0 = 确实没有电流通过）。如果负载明显在工作，重启一次插座再看；仍然如此就是这台芯片只上报了电压
升了版本之后一直没有功率数字	面板还在跑旧的那个进程（新功能在页面里，要重新启动才生效）	关掉面板重开一次；浏览器那页用 `Ctrl-F5` 刷新一下（读的是旧页面）
刚打开页面时功率有数字，过一会儿变成「未读到读数（no reply to `Status 8`）」	旧版面板发了一条不存在的指令（`StatusSNS`），第一次能读到是因为继电器那条查询顺带带回了同一段数据	升级到当前版本并**重启面板**（这个 BUG 在 v3.7 首版里，已修）
想过几秒就刷新一次功率	默认 5 秒是刻意的：同一台插座的读取共用一条队列	用 `--energy 1` 可以调到最小 1 秒，但会让继电器状态读得更慢（见 §9.3）

附录 A MQTT 主题映射

Tasmota 的 `FullTopic` 是 `%prefix%/%topic%/`，默认主题 `tasmota_%06X`，其中 `%06X` 是设备开机打印的 chip id。本机 `DVES_6FCE2C` → `tasmota_6FCE2C`。

方向	主题	载荷 / 说明
发	`cmnd/<topic>/POWER`	`ON / OFF / TOGGLE`
发	`cmnd/<topic>/POWER0`	全部继电器；早期固件的回退路径
发	`cmnd/<topic>/Status`	`0` 查继电器位掩码
发	`cmnd/<topic>/Status`	`7` 查时钟与时区
发	`cmnd/<topic>/Timer<n>`	定时器 JSON，n = 1~16
发	`cmnd/<topic>/Timers`	`0/1/2` 全局关/开/翻转；空载荷 = 列出
发	`cmnd/<topic>/Timezone`	`+8:00`（本机固件拒绝 `+08:00`）
收	`stat/<topic>/RESULT`	命令回执，错误也走这里
收	`stat/<topic>/POWER`	`ON / OFF`
收	`stat/<topic>/STATUS`	`{"Status":{...,"Power":"1"}}`
收	`stat/<topic>/STATUS11`	`{"StatusSTS":{...,"POWER":"ON"}}`
收	`tele/<topic>/LWT`	`Online / Offline`（保留消息）

附录 B 本机实测注意事项

以下都是 2026-09-25
在真机上验证到的、与通用文档不同的地方。脚本已全部处理，记在这里是为了排查时不被误导。

- **状态怎么读**：用 `Status 0`，取 `{"Status":{...}}` 里的 `Power`。**不要**依赖保留消息 —— 设备的 `PowerRetain/StateRetain/StatusRetain` 全为 0，状态类主题不保留，必须主动查询；只有 `tele/.../LWT` 是保留的。
- **别把能耗当状态**：`StatusSNS.ENERGY.Power` 是「瓦特」，和继电器位掩码 `Power` 同名。脚本优先找显式的 `POWER / POWER<n>` 键，且不进入 `ENERGY` 子树，否则 0 瓦会被误读成「继电器 OFF」。
- **千万别用设备的 Client ID**：必须用随机 ID（脚本生成 `wb-mqtt-py-xxxxxx`），填 `DVES_6FCE2C` 会把插座自己顶下线。
- **命令默认不带 retain**：带上后插座每次重连 broker 都会重放最后一条命令，可能莫名其妙自己开关。只有要复刻设备侧设置时才加 `--retain`。
- **设备会主动推状态**：`TelePeriod` 为 300 秒，期间可能来一条 `STATUS11` 带着命令前的旧值；脚本判定失败前会补读一次，避免误报。
- **设备配置当天变过**：早期继电器完全没配（所有 GPIO 未分配、模板无 Relay），只有 `POWER0` 被接受且无回显；重启后继电器配好，`POWER` 正常工作并回显。脚本两条路径都兼容：先发 `POWER`，被拒才回退 `POWER0`。
- **时钟**：NTP 已同步（设备 UTC 与真实 UTC 秒级一致），但时区是固件默认 `+01:00`，需手动改成 `+8:00`。
- **时区写法的坑**：`Timezone +08:00` 会被**静默拒绝**（回 `+00:00`，时区却回落 UTC），必须写 `+8:00`。脚本现在两种写法都收，自动换算并回读 `Status 7` 校验。

— 文档结束 —